VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

An Autonomous Institute Affiliated to University of Mumbai

Department of Computer Engineering

Project Report on

MailedIt: Your Effortless Email Marketing Companion

In partial fulfilment of the Fourth Year, Bachelor of Engineering (B.E.) Degree in Computer Engineering at the University of Mumbai
Academic Year 2025 - 26

By

Priya Anandani D17A / 05

Rishi Daryani D17A/18

Om Goplani D17B/ 68

Project Mentor

Mrs. Geocey Shejy

(A.Y. 2025 - 26)

VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

(An Autonomous Institute Affiliated to University of Mumbai)

Department of Computer Engineering

Certificate

This is to certify that **Priya Anandani (D17A - 05), Rishi Daryani (D17A - 18), & Om Goplani (D17B - 68)** of Fourth Year Computer Engineering studying under the University of Mumbai have satisfactorily completed the project on **"MailedIt: Your Effortless Email Marketing Companion"** as a part of their coursework of PROJECT - II for Semester - VIII under the guidance of their mentor **Mrs. Geocey Shejy** in the year 2025 - 26.

This project report entitled **"MailedIt: Your Effortless Email Marketing Companion"** by Priya Anandani, Rishi Daryani, Om Goplani is approved for the degree of **Bachelor of Engineering** in **Computer Engineering.**

Programme Outcomes	Grade
PO1, PO2, PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PO12 PSO1 & PSO2	

Date: 27th April, 2026

Project Guide: Mrs. Geocey Shejy

Project Report Approval
For
B. E (Computer Engineering)

This project report entitled **"MailedIt: Your Effortless Email Marketing Companion"** by **Priya Anandani (D17A - 05), Rishi Daryani (D17A - 18), & Om Goplani (D17B - 68)** is approved for the degree of **Bachelor of Engineering** in **Computer Engineering.**

Examiners

1. ……

(Internal Examiner name & sign)

2. ……

(External Examiner name & sign)

3. ……

(Head of Department)

4. ……

(Principal)

Date: 27th April, 2026

Place: Chembur, Mumbai

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

- (Priya Anandani - D17A / 05)

- (Rishi Daryani - D17A / 18)

- (Om Goplani - D17B / 68)

Date: 27th April, 2026

Acknowledgement

We are thankful to our college Vivekanand Education Society's Institute of Technology for considering our project and extending help at all stages needed during our work of collecting information regarding the project.

It gives us immense pleasure to express our deep and sincere gratitude to the Head of the Computer Department **Mrs. Geocey Shejy** (Project Guide) for her kind help and valuable advice during the development of project synopsis and for her guidance and suggestions.

We are deeply indebted to our Principal **Dr. (Mrs.) J.M. Nair**, for giving us this valuable opportunity to do this project.

We express our hearty thanks to them for their assistance without which it would have been difficult to finish this project synopsis and project review successfully.

We convey our deep sense of gratitude to all teaching and non-teaching staff for their constant encouragement, support and selfless help throughout the project work. It is a great pleasure to acknowledge the help and suggestion, which we received from the Department of Computer Engineering.

We wish to express our profound thanks to all those who helped us in gathering information about the project. Our families too have provided moral support and encouragement several times.

COURSE OUTCOMES FOR B.E. PROJECT

Learners will be able to,

Course Outcome	Description of the Course Outcome
CO1	Able to apply the relevant engineering concepts, knowledge and skills towards the project.
CO2	Able to identify, formulate and interpret the various relevant research papers and to determine the problem.
CO3	Able to apply the engineering concepts towards designing solutions for the problem.
CO4	Able to interpret the data and datasets to be utilized.
CO5	Able to create, select and apply appropriate technologies, techniques, resources and tools for the project.
CO6	Able to apply ethical, professional policies and principles towards societal, environmental, safety and cultural benefit.
CO7	Able to function effectively as an individual, and as a member of a team, allocating roles with clear lines of responsibility and accountability.
CO8	Able to write effective reports, design documents and make effective presentations.
CO9	Able to apply engineering and management principles to the project as a team member.
CO10	Able to apply the project domain knowledge to sharpen one's competency.
CO11	Able to develop a professional, presentational, balanced and structured approach towards project development.
CO12	Able to adopt skills, languages, environment and platforms for creating innovative solutions for the project.

Index

Chapter No.	Title	Page Number
	Abstract	**1**
Chapter 1	**Introduction**	**2**
1.1	Introduction	2
1.2	Motivation	2
1.3	Problem Definition	3
1.4	Existing Systems	4
1.5	Relevance of the Project	4
1.6	Methodology Used	5
Chapter 2	**Literature Survey**	**6**
2.1	Research Papers a. Abstract of the research paper b. Inference drawn from the paper	7
2.2	Patent Search	10
2.3	Inference Drawn	11
2.4	Comparison with the existing system	11
Chapter 3	**Requirement Gathering for the Proposed System**	**14**
3.1	Introduction to requirement gathering	14
3.2	Functional Requirements	14
3.3	Non-Functional Requirements	15
3.4	Hardware, Software, Tools and Techniques utilized	15
3.5	Constraints	16
Chapter 4	**Proposed Design**	**18**
4.1	Architecture Diagram of system	18
4.2	Modular Diagram Representation of the System	19
4.3	Detailed Design	19
4.4	Project Scheduling & Tracking using Gantt Chart	20

Chapter No.	Title	Page Number
Chapter 5	**Implementation of the Proposed System**	**21**
5.1	Methodology employed for development	21
5.2	Algorithms and flowcharts for the respective modules developed.	22
5.3	Datasets source and utilization	24
Chapter 6	**Testing of the Proposed System**	**25**
6.1	Introduction to Testing	25
6.2	Types of Tests Considered	25
6.3	Various Test Case Scenarios Considered	25
6.4	Inference drawn from test cases	26
Chapter 7	**Conclusion**	**27**
7.1	Screenshots of UI for Respective modules	27
7.2	Performance Evaluation Measures	29
7.3	Input Parameters/ Features Considered	30
7.4	Graphical and Statistical Output	31
7.5	Comparison of results with Existing Systems	31
7.6	Inference Drawn	31
Chapter 8	**Conclusion**	**32**
8.1	Limitations	32
8.2	Conclusion	32
8.3	Future Scope	32
Chapter 9	**References**	**34**
Chapter 10	**Appendix**	**35**

Abstract

This project presents the design and development of a full-stack email marketing platform, MailedIt, aimed at simplifying and optimizing digital campaign management. In today's data-driven landscape, businesses require efficient tools to manage large-scale email campaigns, segment audiences, and analyze performance. MailedIt addresses these challenges by offering an integrated solution that streamlines the entire workflow—from campaign creation to performance tracking—while ensuring scalability, reliability, and ease of use.

The platform is built using modern web technologies, with a Laravel backend handling core functionalities such as authentication, user management, subscriber segmentation, and bulk email processing. It utilizes queue-based job handling to ensure efficient and reliable email delivery under high workloads. The frontend, developed using React, provides a responsive and interactive interface with features like a campaign creation wizard, customizable templates, and an analytics dashboard. This architecture ensures a seamless user experience along with maintainable and scalable system design.

Additionally, MailedIt emphasizes data-driven decision-making by offering insights into key performance metrics such as open rates, click-through rates, and user engagement. The system is designed to adapt to increasing user demands, making it suitable for both small businesses and growing enterprises, while improving overall efficiency and reducing manual effort in email marketing operations.

Chapter 1: Introduction

This chapter introduces the project MailedIt – Your Email Marketing Companion. It presents the motivation behind developing the system, highlights the drawbacks of existing solutions, defines the problem statement, and describes the methodology adopted to design and implement the system. It also explains the relevance and importance of the project in today's digital marketing environment.

1.1 Introduction

MailedIt is a full-stack email marketing platform designed to simplify and streamline the process of creating, managing, and analyzing email campaigns. In an era where effective communication plays a crucial role in business growth, the platform provides marketers with a centralized solution to handle their outreach efforts efficiently. It focuses on reducing the complexity involved in campaign management while offering a modern, intuitive, and performance-driven environment for users to engage with their audience.

The backend of the system is developed using Laravel, which ensures secure data handling, structured business logic, and reliable API services. It supports key functionalities such as user authentication, subscriber management, and campaign processing, along with seamless integration with third-party email services for message delivery. The use of queue-based job processing enhances system performance by handling bulk email operations asynchronously, ensuring scalability and minimal delays even during high-volume campaigns.

On the frontend, the platform is built using ReactJS, providing a dynamic and responsive user interface that enhances overall user experience. It includes essential modules such as campaign creation with guided workflows, subscriber segmentation for targeted communication, customizable email template design, and analytics visualization for tracking performance metrics. The decoupled architecture allows independent development and scaling of frontend and backend components, making the system flexible and maintainable. Overall, MailedIt effectively bridges the gap between simplicity and functionality, enabling users to manage their entire email marketing workflow through one integrated and efficient platform.

1.2 Motivation for the Project

In today's competitive digital era, email marketing remains one of the most effective and widely used tools for customer engagement, brand promotion, and retention. Businesses rely on personalized and targeted email campaigns to communicate with their audience, promote products, and build long-term relationships. However, many existing email marketing platforms often present challenges for small to mid-scale businesses, including high subscription costs, complex user interfaces, and limitations on customization or scalability. These barriers make it difficult for emerging businesses and independent marketers to fully leverage the potential of email marketing.

The motivation behind developing MailedIt stems from the need to address these limitations by providing a more accessible and flexible solution. The platform is envisioned as an open-source and self-hosted alternative that allows users to have complete control over their data, customization, and deployment. By eliminating dependency on costly third-party subscriptions, MailedIt offers a cost-effective approach while still incorporating essential professional features such as campaign management, audience segmentation, and performance tracking. This makes it particularly beneficial for startups, small businesses, and developers seeking a customizable marketing solution.

Additionally, the project serves as a practical exploration of modern full-stack development practices and system design. It involves implementing real-time email delivery mechanisms, integrating external APIs for email services, and managing asynchronous processes using queues for efficient performance. The development of MailedIt also focuses on building a scalable and modular architecture that can handle increasing workloads and adapt to evolving requirements. Overall, the project is driven by the goal of creating a reliable, efficient, and developer-friendly email marketing system while gaining hands-on experience with technologies and concepts relevant to modern web applications.

1.3. Drawback of the Existing System

Existing email marketing platforms, while feature-rich and widely adopted, come with several inherent limitations that can restrict their usability, flexibility, and accessibility—especially for small to mid-scale businesses. One of the most significant challenges is the high subscription cost, where advanced functionalities such as automation, detailed analytics, and audience segmentation are often locked behind expensive premium plans. This creates a financial barrier for startups and individual marketers who may not have the resources to invest in such tools, limiting their ability to compete effectively in the digital space.

Another key limitation is the lack of customization offered by these platforms. Since most of them operate as closed systems, users are unable to modify backend logic, implement custom workflows, or tailor the platform according to specific business requirements. This rigidity is further compounded by the dependency on proprietary ecosystems, where businesses must rely on predefined APIs, interfaces, and service constraints. Such dependency not only reduces flexibility but also makes migration to other platforms difficult, leading to vendor lock-in.

Additionally, data privacy and security concerns play a crucial role in limiting the adoption of third-party platforms. Sensitive user and customer data are typically stored on external servers, raising concerns about data ownership, compliance, and potential misuse. For organizations dealing with confidential information, this can be a major drawback. Moreover, many existing tools suffer from overly complex user interfaces, as they are packed with numerous features that can overwhelm new users. This steep learning curve reduces efficiency and makes it difficult for non-technical users to effectively utilize the platform.

To address these challenges, MailedIt is designed as an open-source, self-hosted, and highly customizable alternative. It empowers users with full control over their data, system configuration, and feature enhancements. By offering a simplified and user-friendly interface along with the flexibility to extend functionalities, MailedIt bridges the gap between powerful capabilities and ease of use, providing a scalable and independent solution for modern email marketing needs.

1.4. Problem Definition

Email marketing systems are required to handle a wide range of critical processes, including audience segmentation, campaign scheduling, content management, performance analysis, and delivery optimization. These functionalities must work together seamlessly to ensure effective communication and high engagement rates. However, most existing solutions struggle to provide a balanced combination of ease of use, scalability, and customization. While some platforms offer advanced features, they are often too complex for beginners, whereas simpler tools lack the flexibility and scalability required for growing business needs. This creates a gap where users are forced to compromise between functionality and usability.

A key challenge lies in integrating all aspects of the email marketing workflow into a single cohesive system. Many platforms provide fragmented solutions, requiring users to rely on multiple tools for designing campaigns, managing subscribers, and analyzing results. This not only increases operational complexity but also leads to inefficiencies in data handling and decision-making. Additionally, scalability becomes a concern when dealing with large volumes of subscribers and emails, as systems must ensure reliable delivery without performance degradation. Security is another critical factor, as sensitive user and customer data must be protected through proper authentication, authorization, and data handling mechanisms.

The problem, therefore, is to design and develop a full-stack web application that unifies the entire email marketing process into one integrated platform. The system should enable users to efficiently manage everything from content creation and campaign execution to real-time analytics and reporting. It must be designed with a scalable architecture to handle increasing workloads, while also maintaining a user-friendly interface that caters to both technical and non-technical users. By addressing these challenges, the proposed solution aims to deliver a secure, flexible, and intuitive platform that enhances productivity and simplifies email marketing operations for users of all experience levels.

1.5. Relevance of the Project

The MailedIt project holds strong relevance in the domain of digital marketing automation, especially in an era where businesses increasingly rely on personalized and data-driven communication strategies. As organizations shift toward targeted marketing approaches, there is a growing need for platforms that not only provide powerful features but also ensure cost efficiency, transparency, and full control over data and operations. Many existing tools fail to meet these requirements simultaneously, which highlights the importance of developing solutions like MailedIt that balance functionality with accessibility. By offering a self-managed and customizable system, the project aligns with current industry demands for flexible and scalable marketing technologies.

From an academic and practical perspective, the project is highly valuable as it enables students and developers to gain hands-on experience with modern web development practices and real-world system design. One of the key aspects is full-stack development, where learners work with Laravel for backend services and React for frontend interfaces, understanding how both layers interact to build a cohesive application. Additionally, the project involves integration with third-party APIs, particularly for email delivery services, which helps in understanding external service communication and API handling—an essential skill in today's interconnected software ecosystem.

Another important area of learning is the implementation of background processing using queues and workers, which is critical for handling time-consuming tasks like bulk email sending without affecting system performance. This introduced us to concepts such as asynchronous processing, job scheduling, and system optimization. Furthermore, the project emphasizes the design of responsive and data-driven user interfaces, where users can visualize analytics, manage campaigns, and interact with the system intuitively. Overall, MailedIt not only addresses real-world challenges in email marketing but also serves as a comprehensive learning platform for mastering key technologies and architectural concepts relevant to modern web applications.

1.6. Methodology Used

The development of MailedIt follows a component-based and decoupled architecture, ensuring that the system remains modular, scalable, and easy to maintain. This approach allows individual components of the system to be developed, tested, and deployed independently, improving flexibility and reducing the complexity of future enhancements. By separating concerns between the frontend, backend, and data layers, the platform achieves better performance, maintainability, and the ability to scale efficiently as user demand grows.

Backend Setup (Laravel)

The backend is implemented using Laravel, which serves as the core engine of the application. It is responsible for creating secure and structured RESTful APIs that handle authentication, user management, subscriber data, and campaign-related operations. To ensure efficient handling of bulk email processes, a queue-based email dispatch system is implemented. This allows time-consuming tasks such as sending large volumes of emails to be processed asynchronously, improving system responsiveness and reliability. Additionally, the backend integrates with third-party email service providers, enabling real-time email delivery, tracking, and status monitoring (such as sent, delivered, or failed emails), which is crucial for campaign effectiveness analysis.

Frontend Development (ReactJS)

The frontend is developed using ReactJS, focusing on building a highly interactive and user-friendly interface. A component-based approach is used to create reusable UI elements, improving consistency and reducing development effort. Client-side routing ensures smooth navigation between different modules without page reloads.

Key features include a campaign creation wizard, which guides users step-by-step through the process of designing and launching campaigns, and an email template editor that allows customization of content. Furthermore, the platform incorporates data visualization tools using charting libraries to present analytics such as open rates, click-through rates, and engagement trends in a clear and visually appealing manner.

Database and Cloud Integration

For data management, MySQL is used as the primary relational database to store structured data such as users, campaigns, and subscriber information. To enhance performance and support asynchronous operations, Redis is utilized for caching and managing background jobs. This combination ensures faster data retrieval, reduced database load, and efficient execution of queued tasks. The architecture is designed to be cloud-compatible, allowing easy deployment and scalability in distributed environments.

Testing and Evaluation

To ensure system reliability and performance, comprehensive testing strategies are applied. Unit testing is conducted to validate individual components, while integration testing ensures smooth interaction between different modules of the system. Additionally, performance testing is carried out to evaluate email delivery speed, queue processing efficiency, and API response times under varying workloads. This rigorous evaluation helps identify bottlenecks and ensures that the platform performs consistently, even during high-demand scenarios.

Overall, the structured development methodology adopted in MailedIt ensures a robust, scalable, and efficient system capable of meeting modern email marketing requirements while maintaining high performance and usability.

Chapter 2: Literature Survey

A. Brief Overview of Literature Survey

The literature survey for the MailedIt project focuses on recent research and industry insights related to email systems, phishing detection, spam filtering, and marketing automation. Various research papers from the past few years highlight the increasing complexity of email security and the need for intelligent, adaptive systems. Studies emphasize the importance of combining multiple features such as email content, headers, URLs, and metadata to improve the accuracy of detecting malicious or spam emails. Advanced approaches like transformer-based NLP models, machine learning algorithms, and hybrid detection techniques demonstrate how integrating semantic and structural analysis can significantly enhance email classification and threat detection capabilities.

Additionally, research exploring adaptive and self-evolving systems shows that modern email threats continuously evolve, requiring dynamic and learning-based defense mechanisms. The use of updated datasets and realistic testing environments is also highlighted as essential for building reliable and generalizable systems. Comparative studies on machine learning models further provide benchmarks for accuracy and performance, offering valuable insights for designing efficient filtering and analytics modules within MailedIt.

Apart from research papers, books, articles, and industry reports contribute to understanding the fundamentals of digital marketing, personalization, and user engagement strategies. These sources emphasize the growing role of data-driven decision-making, audience segmentation, and automation in email marketing. Interactions with domain experts and references to existing patents further provide practical insights into system design, template generation, and campaign management. Overall, the literature survey helps in shaping MailedIt as a secure, scalable, and intelligent platform by combining technical advancements with real-world marketing requirements.

B. Related Works

2.1 Research Papers Referred

2.1.1 A Combined Feature Selection Approach for Malicious Email Detection (2025)

a. Abstract of the Research Paper

This research focuses on improving malicious email detection by addressing two major limitations in existing systems: the lack of updated and unified datasets, and the reliance on limited feature extraction techniques. Traditional approaches primarily depend on static features or body-text analysis, which often fail to capture the complexity of modern phishing and spoofing attacks. The authors propose a combined feature selection approach that integrates multiple dimensions of email data, including protocol-level information, header attributes, body content, and both static and dynamic features. This comprehensive approach enhances the system's ability to detect a wider range of threats and improves robustness against evolving attack patterns.

b. Inference Drawn from the Paper

The study demonstrates that hybrid feature extraction significantly enhances detection accuracy and system reliability. For MailedIt, this suggests the importance of incorporating diverse data sources and multi-layered analysis techniques to build an effective spam and phishing detection mechanism.

2.1.2 Dual-Path Phishing Detection: Integrating Transformer-Based NLP with Structural URL Analysis (2025)

a. Abstract of the Research Paper

This paper introduces a dual-path phishing detection model that combines advanced Natural Language Processing (NLP) with structural URL analysis. The system uses transformer-based models such as DistilBERT to understand the semantic meaning of email content, while simultaneously applying character-level TF-IDF techniques to analyze URL structures embedded within emails. By integrating these two approaches, the model achieves a more comprehensive understanding of both textual and structural elements, leading to improved detection performance and better interpretability of results.

b. Inference Drawn from the Paper

The research highlights the effectiveness of combining semantic and structural analysis for phishing detection. This approach can be applied in MailedIt to enhance security by analyzing both email content and embedded links, thereby reducing the chances of malicious emails bypassing the system.

2.1.3 EvoMail: Self-Evolving Cognitive Agents for Adaptive Spam and Phishing Email Defense (2025)

a. Abstract of the Research Paper

EvoMail proposes an innovative framework based on self-evolving cognitive agents designed to counter continuously changing phishing and spam threats. The system includes red-team agents that simulate new phishing strategies and blue-team agents that learn and adapt to these attacks. By using heterogeneous email graphs that combine multiple data types such as content, metadata, URLs, and attachments, the system maintains high detection accuracy even as threats evolve over time.

b. Inference Drawn from the Paper

This work emphasizes the importance of adaptive and intelligent systems in cybersecurity. For MailedIt, it provides insight into incorporating self-learning or AI-driven mechanisms that can continuously update and improve spam detection strategies, ensuring long-term system effectiveness.

2.1.4 E-PhishGen: Unlocking Novel Research in Phishing Email Detection (2025)

a. Abstract of the Research Paper

This paper critically examines the limitations of existing phishing detection models, particularly their dependence on outdated datasets. It introduces a new multilingual dataset, E-PhishLLM, which contains recent and realistic phishing email samples. The authors argue that many models achieve high accuracy only because they are tested on obsolete data, and they stress the importance of evaluating systems using modern, real-world datasets to ensure reliability and generalization.

b. Inference Drawn from the Paper

The study highlights the necessity of using updated datasets for testing and validation. This insight guides MailedIt to incorporate realistic data scenarios when evaluating its spam filtering and analytics modules, ensuring better real-world performance.

2.1.5 Detecting Email Phishing Using Machine Learning Algorithms (2025)

a. Abstract of the Research Paper

This research compares the performance of various machine learning algorithms, including Naïve Bayes, Random Forest, Support Vector Machines, and K-Nearest Neighbors, on a dataset of over 18,000 emails. The study finds that Random Forest achieves the highest accuracy (98.3%) along with strong F1 scores. However, it also identifies challenges such as false negatives, particularly in cases where phishing emails are cleverly disguised to appear legitimate.

b. Inference Drawn from the Paper

The paper provides valuable benchmarks for evaluating classification models. For MailedIt, it highlights the importance of selecting appropriate algorithms and minimizing false negatives to ensure robust email security and reliable spam detection.

2.1.6 Evaluating Spam Filters and Stylometric Detection of AI-Generated Phishing Emails (2025)

a. Abstract of the Research Paper

This study evaluates how existing spam filters perform against AI-generated phishing emails. It uses stylometric analysis involving 60 linguistic features and tests various machine learning models such as Logistic Regression, SVM, Random Forest, and XGBoost. The results show that XGBoost achieves high accuracy (approximately 96%) and strong AUC scores. The research also identifies key linguistic patterns that distinguish AI-generated phishing emails from genuine ones.

b. Inference Drawn from the Paper

The findings reveal that even advanced spam filters can be bypassed by sophisticated AI-generated emails. This highlights the need for more advanced detection strategies in MailedIt, such as incorporating behavioral and linguistic analysis to strengthen its security framework.

Overall, these related works provide a strong foundation for understanding modern challenges and advancements in email security and marketing systems. They emphasize the importance of hybrid models, adaptive learning, and real-world data evaluation, all of which influence the design and future scope of the MailedIt platform.

2.2 Patent Search

A preliminary patent search was conducted to explore existing innovations related to email marketing systems, campaign management, and automated email processing. This step ensured that the proposed project MailedIt is original and does not conflict with existing patented solutions. The following patents were studied as part of the research:

1. Patent Title: Email Management System (US 2024/0333675 A1, 2024)
Summary:
This patent describes a system that introduces a **unified contact ID**, allowing multiple contact records of a single user to be merged into one. This improves data consistency and enables seamless automation across marketing workflows. It enhances campaign targeting by maintaining a single, accurate subscriber profile.

Link: https://patents.google.com/patent/US20240333675A1/en

2. Patent Title: Smart Electronic Mail and Messaging Templates (US 10984186 B2, 2021)
Summary:
This patent focuses on the automatic generation of email templates by identifying common patterns across multiple emails. It creates a reusable base template and allows dynamic customization of specific sections. This approach reduces manual effort and ensures consistency in email campaigns.

Link: https://patents.google.com/patent/US10984186B2/en

3. Patent Title: Email Campaign Auditing System (US Patent Application)
Summary:
This patent describes a system designed to audit and evaluate email marketing campaigns. It checks factors such as email deliverability, server behavior, spam compliance, and link validation. The system ensures that campaigns meet required standards before being sent, improving reliability and performance.

Link: https://patents.google.com/

4. Patent Title: Systems and Methods for Managing Email Campaigns and Analytics (US 2020/0175070 A1, 2020)
Summary:
This patent presents a system for managing email campaigns along with tracking and analyzing user engagement metrics such as open rates, click-through rates, and conversions. It integrates analytics with campaign execution to provide insights for improving marketing strategies.

Link: https://patents.google.com/patent/US20200175070A1/en

These patents provided valuable insights into modern email marketing technologies, including **contact management, template automation, campaign auditing, and analytics tracking**. They guided the design and feature implementation of MailedIt while ensuring originality and alignment with current industry practices.

2.3 Inference Drawn

The literature and patent studies reviewed in the previous sections highlight the rapid evolution of email systems, particularly in the areas of security, automation, and scalability. A key observation is that modern email platforms are no longer limited to basic communication but have evolved into intelligent systems that integrate data analysis, personalization, and threat detection. Research works emphasize the importance of combining multiple techniques—such as machine learning, NLP, and structural analysis—to improve the accuracy of spam and phishing detection. This indicates that a multi-layered approach is essential for building robust and reliable email marketing platforms.

Another important insight is the growing need for adaptive and data-driven systems. Several studies demonstrate that static models are insufficient to handle evolving threats like AI-generated phishing emails. Instead, systems must continuously learn and adapt using updated datasets and real-world scenarios. This highlights the importance of incorporating scalable architectures, efficient data processing mechanisms, and continuous evaluation strategies in MailedIt to ensure long-term effectiveness and reliability.

From the patent analysis, it is evident that industry solutions focus heavily on automation, template optimization, and efficient data management. Features such as unified contact profiles, intelligent template generation, and campaign auditing mechanisms are becoming standard in modern systems. These innovations provide valuable guidance for enhancing MailedIt by improving user experience, reducing redundancy, and ensuring better campaign performance and compliance.

Overall, the combined insights from literature and patents emphasize the need for a system that balances functionality, usability, and flexibility. They reinforce the importance of designing MailedIt as a modular, scalable, and secure platform capable of handling end-to-end email marketing workflows. These findings will guide the architectural decisions, feature implementation, and performance optimization strategies in the subsequent stages of the project.

2.4 Comparison with the existing system

Existing email marketing platforms, although widely used, often exhibit several limitations in terms of cost, flexibility, scalability, and user control. Most traditional systems are built as closed, all-in-one solutions where features are tightly integrated and controlled by the service provider. While these platforms offer convenience, they restrict customization and impose dependency on proprietary ecosystems. In contrast, the proposed MailedIt system adopts a more flexible, modular, and user-centric approach, addressing these limitations effectively.

The following table presents a comparative analysis between existing email marketing systems and the proposed MailedIt platform:

Feature	Existing Systems (MailChimp, Brevo, Active Campaign)	Proposed MailedIt System
Architecture	Closed, tightly controlled platforms	Decoupled, modular full-stack architecture
Cost	Subscription-based, expensive for advanced features	Open-source and self-hosted, cost-effective
Customization	Limited customization, fixed workflows	Highly customizable with extendable backend logic
Data Control	Data stored on third-party servers	Full control with self-hosted data management
Scalability	Limited by provider plans and infrastructure	Scalable using queues, caching, and modular design
User Interface	Feature-heavy and complex for beginners	Simple, intuitive, and user-friendly interface
Email Processing	Often synchronous or limited optimization	Asynchronous processing using queues and workers
Integration	Restricted APIs and ecosystem lock-in	Flexible integration with third-party APIs
Analytics	Basic or premium-restricted analytics	Built-in analytics dashboard with real-time insights

Table 1. Comparison with Existing systems

One of the key advantages of MailedIt lies in its decoupled architecture, where the frontend and backend operate independently, allowing better scalability and maintainability. The use of queue-based background processing ensures efficient handling of bulk email delivery without affecting system performance. Unlike traditional systems that rely heavily on synchronous operations, MailedIt supports asynchronous workflows, improving responsiveness and reliability.

Additionally, MailedIt provides greater transparency and control over data, which is a major concern in existing platforms where user data is stored and managed by third-party providers. By enabling self-hosting, the system ensures better data privacy and compliance. The platform also emphasizes simplicity and usability, overcoming the complexity often seen in existing tools that can overwhelm new users.

Furthermore, MailedIt supports flexible API integration, allowing it to connect with various external services without being restricted to a single ecosystem. This enhances extensibility and future scalability. Combined with built-in analytics and campaign management features, the system provides a complete, end-to-end solution for email marketing.

In summary, the proposed system represents a significant improvement over existing platforms by offering a balance between functionality, flexibility, cost-efficiency, and user control. It transforms the traditional rigid approach into a more dynamic and scalable solution, making it suitable for modern digital marketing requirements.

Chapter 3: Requirement Gathering for the Proposed System

3.1 Introduction to requirement gathering

Requirement gathering is a crucial phase in the system development life cycle, focused on identifying and understanding the needs of users and stakeholders for the MailedIt platform. This process involves defining both functional requirements (such as campaign creation, subscriber management, and analytics) and non-functional requirements (such as scalability, performance, security, and usability). The goal is to design a system that efficiently supports the complete email marketing workflow while ensuring a smooth and user-friendly experience.

The requirements for MailedIt are derived through analysis of existing platforms, research studies, and inputs from domain experts. The system aims to provide a centralized, cost-effective, and customizable solution for managing email campaigns, with features like bulk email handling, third-party API integration, and secure data management. These insights help in defining a robust and scalable architecture that meets real-world marketing and technical needs.

3.2 Functional Requirements

Functional requirements define what the system is expected to perform. These are the core features of *MailedIt* that ensure smooth operation and user satisfaction.

1. **User Authentication and Role Management:**
    - Secure login and registration using Laravel Sanctum.
    - Role-based access for Admin, Marketer, and Viewer.
2. **Subscriber and Audience Management:**
    - Add, edit, or import subscribers via CSV.
    - Segment audiences based on location, behavior, or custom tags.
3. **Campaign Management:**
    - Create, schedule, and send email campaigns.
    - Real-time monitoring of sending progress and delivery status.
4. **Template Builder:**
    - Drag-and-drop or rich-text editor for email templates.
    - Save, edit, or reuse templates for future campaigns.
5. **Analytics and Reporting:**
    - Track open rates, click-through rates, and bounce data.
    - Display performance insights via charts and graphs.
6. **Queue-Based Email Dispatch:**
    - Use Laravel queues and Redis for reliable email sending.
    - Retry mechanism for failed email jobs.
7. **User Dashboard:**
    - Centralized dashboard showing campaign statistics and recent activities.

3.3 Non-Functional Requirements

Non-functional requirements ensure that the system is reliable, efficient, and user-friendly.

1. **Performance:**
   The system must handle large subscriber lists efficiently using queue-based processing. This ensures that bulk email operations are executed without delays or system slowdowns.
2. **Scalability:**
   The platform should support multiple users and campaigns running simultaneously, with the ability to scale as the number of users and data increases.
3. **Security:**
   User credentials and subscriber data must be protected using encryption techniques and secure authentication mechanisms to prevent unauthorized access.
4. **Usability:**
   The interface should be intuitive, easy to navigate, and responsive across all devices, ensuring a smooth experience for both technical and non-technical users.
5. **Availability:**
   The system should maintain minimal downtime, with APIs designed to achieve high availability (around 99% uptime) for uninterrupted operations.
6. **Maintainability:**
   The codebase should be modular, reusable, and well-documented, making it easier to update, debug, and extend the system in the future.
7. **Compatibility:**
   The application should be compatible with all modern web browsers and provide a responsive design that adapts seamlessly to different screen sizes and devices.

3.4 Hardware, Software, Tools, and Techniques Utilized Hardware

Hardware Requirements

The development and testing of the MailedIt system requires a standard computing environment capable of handling full-stack application development and background processing tasks.

1. **Processor:** Intel Core i5 or higher (or equivalent) to support backend processing and frontend rendering efficiently.
2. **RAM:** Minimum 8 GB (16 GB recommended) to handle development tools, servers, and queue workers simultaneously.
3. **Storage:** At least 100 GB HDD/SSD for storing project files, databases, and dependencies.
4. **Internet Connectivity:** Stable internet connection required for API integrations, package installations, and email service communication

Software Requirements

The system is built using modern web technologies that support scalability and performance.

1. **Operating System:** Windows, Linux, or macOS
2. **Backend:** Laravel 12, PHP 8.3+, Composer
3. **Frontend:** React.js, Node.js, npm/yarn
4. **Database:** MySQL for structured data storage
5. **Cache/Queue:** Redis for caching and background job processing
6. **Web Server:** Apache or Nginx

Techniques Utilized

The development of MailedIt incorporates several modern software engineering techniques to ensure efficiency and scalability:

1. **API-Driven Architecture:** RESTful APIs enable seamless communication between frontend and backend.
2. **Queue-Based Processing:** Laravel queues are used for asynchronous email dispatch, improving performance.
3. **Component-Based Frontend:** Reusable React components enhance maintainability and UI consistency.
4. **Data Visualization:** Charting libraries are used to present campaign analytics in an intuitive manner.
5. **Authentication & Authorization:** Secure access is implemented using Laravel Sanctum.

Tools Utilized

Category	Tools / Technologies
Backend	Laravel 12, Composer
Frontend	React.js, Node.js
Database	MySQL
Version Control	Git & GitHub
API Client	Axios
Queue / Cache	Redis, Laravel Horizon
Deployment	Nginx / Apache

Table 2. Tools Utilized

3.5 Constraints

The development and operation of the MailedIt system are influenced by several practical and technical constraints that impact its performance, scalability, and reliability.

1. **Email Delivery Limits:**
   The system relies on third-party email service providers such as Mailgun or SendGrid for sending emails. These services impose limits on the number of emails sent per day or per second, which may affect large-scale campaign execution.
2. **Internet Connectivity:**
   A stable internet connection is essential for scheduling campaigns, communicating with external APIs, and ensuring timely email delivery. Network instability can lead to delays or failures in campaign execution.
3. **Data Privacy:**
   The system must comply with global anti-spam and data protection regulations such as CAN-SPAM and GDPR. This requires careful handling of user data, consent management, and secure storage practices.
4. **Hardware Resources:**
   System performance is dependent on available hardware resources such as CPU, memory, and storage. Limited resources may affect queue processing speed, analytics computation, and overall system responsiveness, especially under heavy workloads.
5. **Third-Party Service Reliability:**
   The availability and performance of external APIs directly impact the system's reliability. Any downtime or latency in third-party services can disrupt email delivery and tracking functionalities.

Chapter 4: Proposed Design

4.1 Architecture Diagram of the System

The MailedIt system architecture uses a client–server model where all components interact seamlessly for efficient email marketing. The React.js frontend enables users to log in, create campaigns, and view analytics while communicating with the backend through Axios APIs. The Laravel backend handles authentication, business logic, and campaign scheduling, with all data stored in the MySQL database.

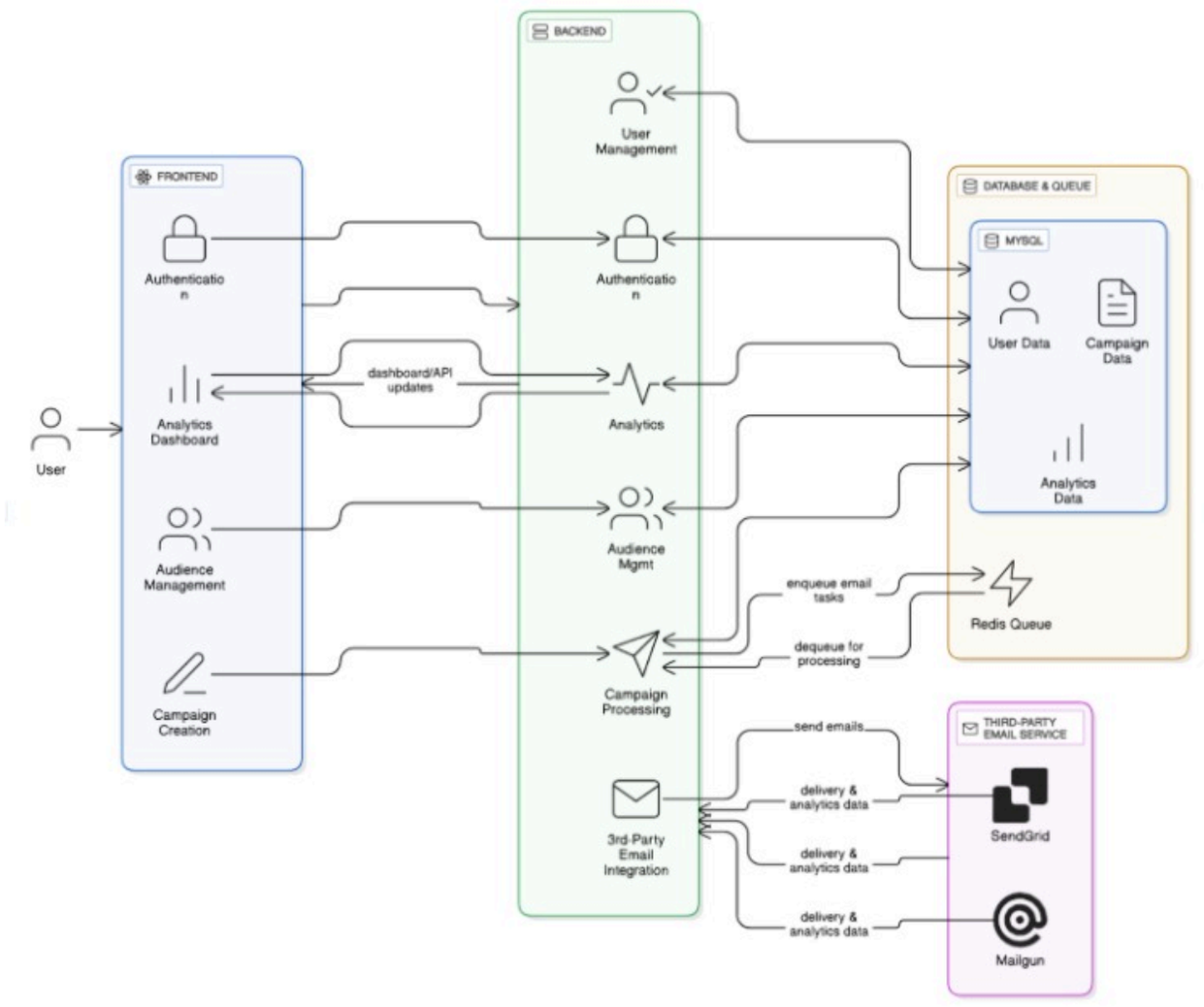

Fig 1. Architecture of MailedIt System

A Redis-based queue system manages background tasks like bulk email sending and report generation. Email delivery is handled by third-party services such as Mailgun or SendGrid, while the Analytics Engine tracks metrics like open, click, and bounce rates to help users evaluate campaign performance.

4.2 Modular Diagram Representation of the proposed system

The modular diagram of MailedIt shows how the system is divided into six main modules — User Management, Audience Management, Campaign Management, Automation, Analysis, and Integration & Queue. Each module performs a specific function: managing users, handling subscriber lists, creating and scheduling campaigns, automating workflows, tracking performance analytics, and integrating with third-party email services. Together, they ensure smooth communication, scalability, and efficient email marketing operations across the platform.

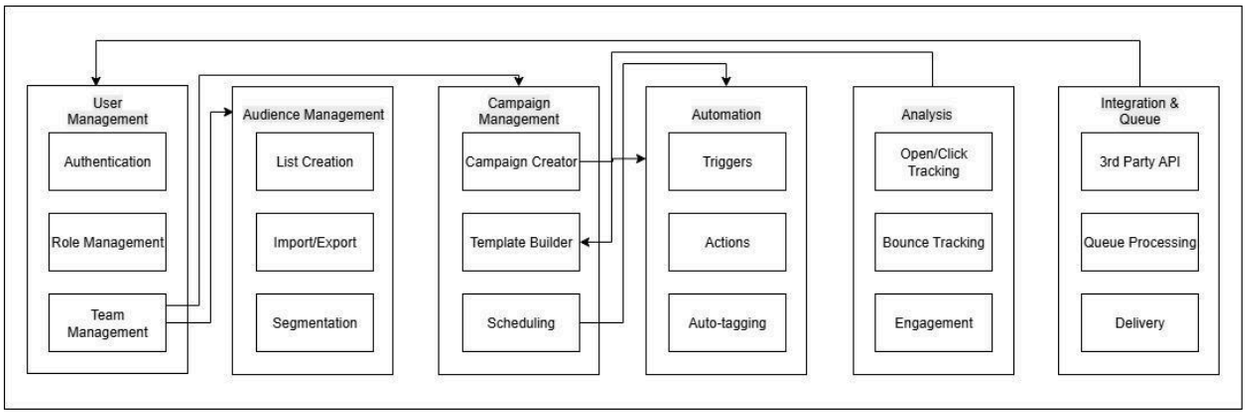

Fig 2. Modular Diagram of MailedIt System

4.3 Detailed Design

The detailed design of the MailedIt system focuses on defining the internal architecture, module interactions, and data flow to ensure efficient and scalable email marketing operations. The system follows a client–server architecture with a clear separation between frontend, backend, database, and external services. Each component is designed to handle specific responsibilities while maintaining seamless communication through APIs and asynchronous processing.

The frontend, developed using React.js, provides an interactive interface for users to perform actions such as authentication, audience management, campaign creation, and analytics visualization. These requests are sent to the Laravel backend, which acts as the core processing unit. The backend handles business logic, including user authentication, campaign scheduling, audience segmentation, and analytics processing. Data is stored in a MySQL database, which maintains structured records of users, campaigns, and analytics data.

A key component of the system design is the queue-based processing mechanism implemented using Redis. When a campaign is scheduled, email tasks are pushed into the queue, allowing asynchronous processing. This ensures that bulk email delivery does not affect system performance. The backend then processes these queued jobs and integrates with third-party email services such as Mailgun or SendGrid for actual email delivery. These services return delivery status and engagement data, which is further processed and stored for analytics.

The system is divided into multiple modules, including User Management, Audience Management, Campaign Management, Automation, Analytics, and Integration & Queue. Each module operates independently but communicates through well-defined interfaces, ensuring modularity and scalability. The analytics module tracks key performance indicators such as open rates, click-through rates, and bounce rates, helping users evaluate campaign effectiveness.

The data flow design ensures smooth movement of information across the system. At a high level, user inputs such as login, campaign creation, and analytics requests are processed by the backend and result in outputs like reports, email delivery confirmations, and dashboards. At deeper levels, the system manages interactions between internal modules and external APIs, ensuring efficient data processing and real-time updates.

Finally, the flow of operations follows a structured sequence: a user logs into the system, creates and schedules a campaign, which is then added to the queue for processing. The backend sends emails via third-party services, and the analytics module tracks delivery and engagement metrics. This structured and modular design ensures that MailedIt is scalable, efficient, and capable of handling real-world email marketing requirements.

4.4 Project Scheduling & Tracking using Gantt Chart

The Gantt chart shows the MailedIt project timeline from September 2025 to January 2026, divided into key phases. It starts with requirement analysis and design, followed by backend and frontend development.

The later stages include integration, testing, and documentation. Some phases overlap to ensure efficient use of time and timely project completion.

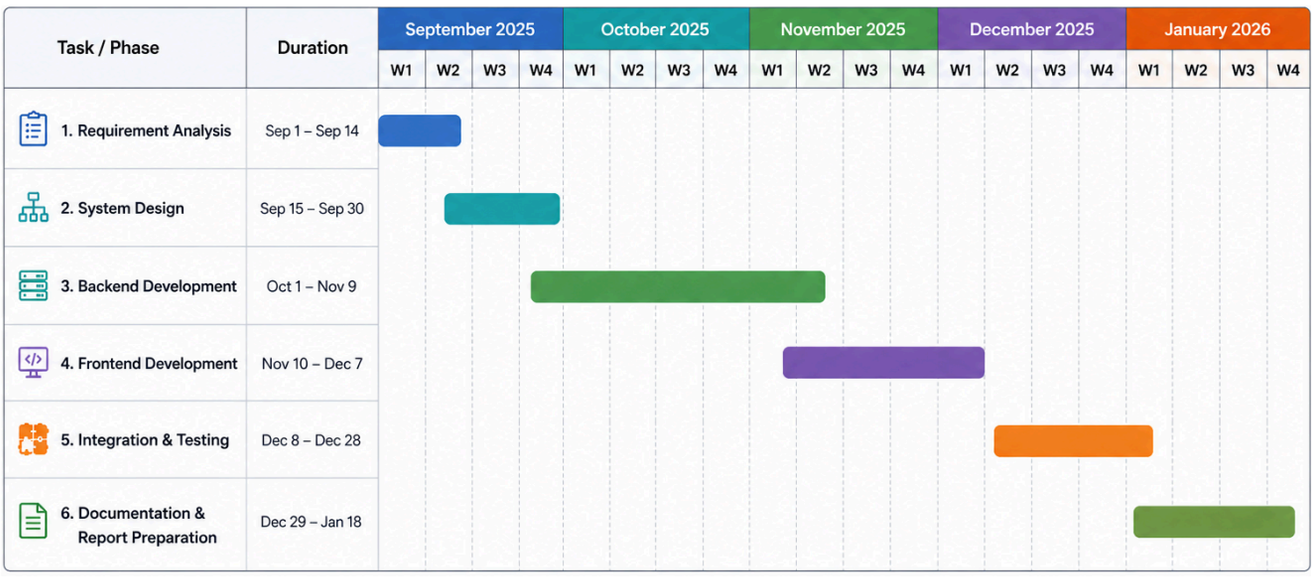

Fig 3. Gantt Chart for MailedIt

Chapter 5: Implementation of the Proposed System

5.1. Methodology employed for development

The development of the MailedIt system follows a full-stack, modular, and API-driven architecture, designed to ensure scalability, maintainability, and efficient email processing. The methodology focuses on separating the system into distinct layers—frontend, backend, database, and third-party integrations—while enabling seamless communication through RESTful APIs. This approach aligns with modern development practices, allowing independent development, testing, and deployment of system components.

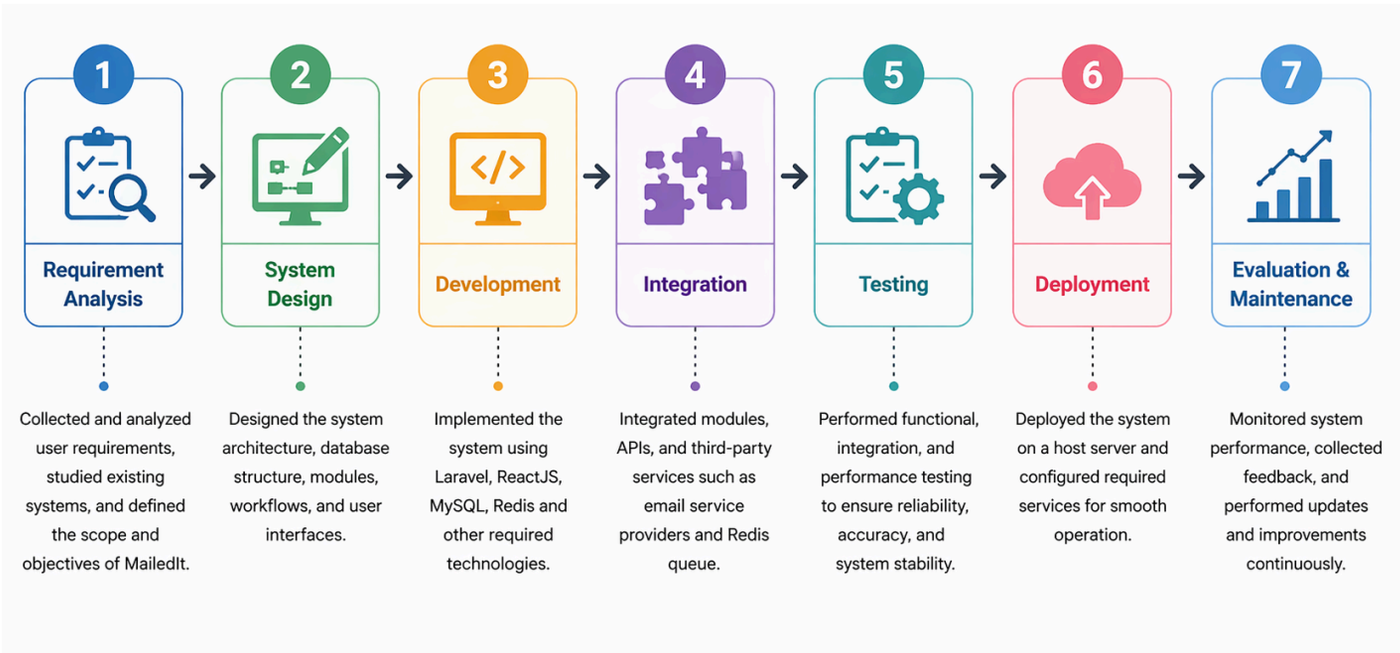

Fig 4. Methodology Architecture

1. **Client Interface (Frontend)**

The React.js frontend serves as the user interface where users perform actions such as login, campaign creation, audience management, and analytics viewing. It communicates with the backend through API calls, ensuring a smooth and interactive user experience while abstracting backend complexity.

2. **Backend Layer (Laravel APIs)**

The Laravel backend acts as the core processing unit, handling authentication, business logic, campaign management, and analytics processing. It exposes secure APIs for communication and ensures proper validation, authorization, and data handling.

3. **Queue-Based Processing System**

A key aspect of the methodology is the use of Redis-based queues for asynchronous processing. Email campaigns are added to the queue and processed in the background, enabling efficient handling of bulk email delivery without affecting system performance.

4. **Third-Party Integration**

The system integrates with external email service providers such as Mailgun and SendGrid for email delivery. These services handle large-scale email sending and provide delivery status and engagement data, which is used for analytics.

5. **Database and Data Management**

MySQL is used for storing structured data such as user information, campaign details, and analytics. Efficient data handling ensures reliability and supports real-time insights for users.

6. **Development and Deployment Practices**

The system follows modular development with version control using Git, enabling collaborative development and easier maintenance. Testing, debugging, and performance monitoring are carried out to ensure system reliability. This methodology supports scalability, allowing the system to handle increasing users and campaigns efficiently.

Overall, the adopted methodology ensures that MailedIt is a scalable, efficient, and maintainable platform, capable of handling end-to-end email marketing workflows with high performance and reliability.

5.2 Algorithms and flowcharts for the system developed

a. Flowchart of System

The flowchart of the MailedIt system represents the complete workflow of an email marketing campaign, starting from user interaction to email delivery and analytics tracking. It illustrates how different modules interact to ensure smooth and efficient campaign execution.

System Workflow Explanation:

1. **User Login:**
   The process begins when the user logs into the system using secure authentication.
2. **Campaign Creation:**
   The user creates an email campaign by selecting templates, adding content, and defining the target audience.
3. **Audience Selection & Segmentation:**
   The system retrieves subscriber data and applies segmentation rules to target the appropriate audience.
4. **Campaign Scheduling:**
   The user schedules the campaign for immediate or future delivery.
5. **Queue Processing:**
   The campaign is added to the queue, where email jobs are processed asynchronously using Redis queues.
6. **Email Delivery:**
   The backend sends emails through third-party services such as Mailgun or SendGrid, ensuring reliable delivery.

7. **Delivery Confirmation:**
   The system receives delivery status updates (sent, delivered, failed) from the email service providers.
8. **Analytics Tracking:**
   The analytics module tracks user engagement metrics such as open rates, click rates, and bounce rates.
9. **Report Generation:**
   The system generates reports and displays them on the analytics dashboard for user insights.
10. **End of Process:**
    The workflow completes after analytics are updated and displayed to the user.

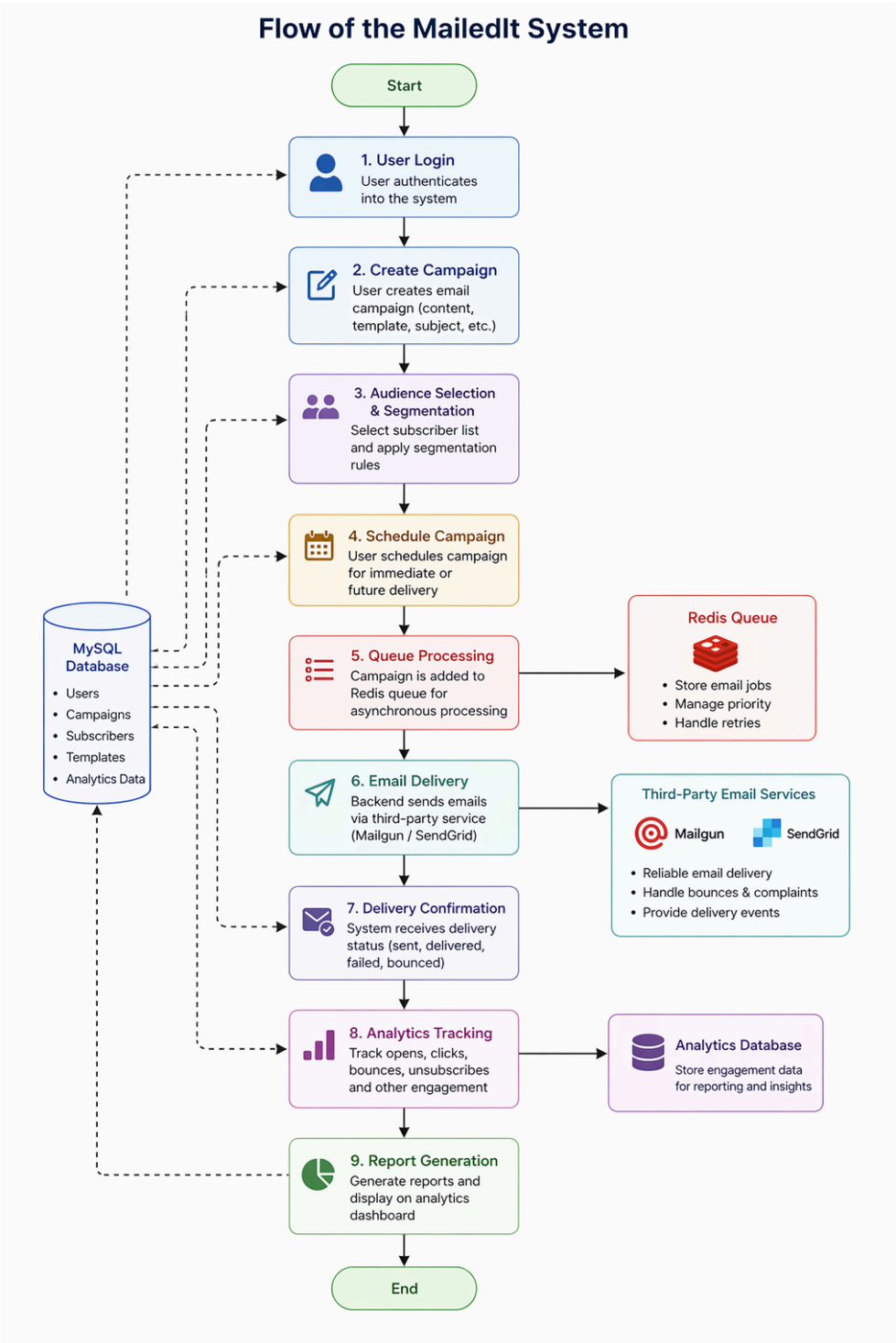

Fig 5. Flow of the System

b. Algorithm for Campaign Processing

1. Start
2. User logs into the system
3. Input campaign details (content, audience, schedule)
4. Validate input data
5. Store campaign details in database
6. Fetch subscriber list
7. Segment audience based on filters
8. Push email tasks to queue
9. For each email in queue:
    a. Fetch email content
    b. Send email via third-party API
    c. Receive delivery response
    d. Store delivery status
10. Collect engagement data (open, click, bounce)
11. Update analytics database
12. Display analytics on dashboard
13. End

Overall, the flowchart and algorithm demonstrate a structured, step-by-step workflow of the MailedIt system, ensuring efficient campaign management, reliable email delivery, and real-time analytics tracking.

5.3 Datasets source and utilization

In the MailedIt project, no external or pre-existing datasets were used. Instead, the system operates on dynamically generated and user-driven data, which closely simulates real-world email marketing scenarios. The data is primarily created through user interactions such as campaign creation, subscriber addition, audience segmentation, and email engagement activities. This approach ensures that the system works with realistic and continuously evolving data rather than static datasets.

The data stored and processed within the system includes:

1. **User Data:** Information related to user accounts and authentication
2. **Subscriber Data:** Email lists, segmentation details, and audience attributes
3. **Campaign Data:** Email content, templates, scheduling information
4. **Analytics Data:** Metrics such as open rates, click rates, bounce rates, and engagement

Additionally, synthetic and test data were generated during development to validate system functionality, including bulk email processing, queue handling, and analytics tracking. This data was structured similarly to real-world inputs, ensuring consistency and reliability during testing.

Chapter 6: Testing of the Proposed System

6.1 Introduction to Testing

Testing is a critical phase in the software development lifecycle that ensures the system functions correctly, efficiently, and reliably. It involves verifying that all modules of the MailedIt platform perform as expected and meet both functional and non-functional requirements. The objective of testing is to identify and eliminate bugs, validate system behavior under different conditions, and ensure a smooth user experience.

For MailedIt, testing focuses on validating key functionalities such as user authentication, campaign creation, email delivery, queue processing, and analytics tracking. It also ensures that the system can handle real-world scenarios like bulk email sending and concurrent user operations without performance degradation.

6.2 Types of Tests Considered

The following types of testing were performed to ensure system reliability:

1. **Unit Testing:**
   Individual modules such as authentication, campaign creation, and API endpoints were tested independently to verify correctness.
2. **Integration Testing:**
   Interaction between modules (e.g., frontend–backend communication, backend–third-party API integration) was tested to ensure seamless data flow.
3. **Functional Testing:**
   The system was tested against defined requirements to ensure all features like campaign scheduling, audience management, and analytics work as expected.
4. **Performance Testing:**
   The system's ability to handle bulk email processing and queue operations was evaluated under different loads.
5. **Usability Testing:**
   The user interface was tested for ease of use, responsiveness, and accessibility across devices.
6. **API Testing:**
   Backend APIs were tested using tools like Postman to verify request-response cycles, validation, and error handling.

6.3 Various Test Case Scenarios Considered

Several real-world scenarios were tested to validate system functionality:

1. **User Authentication:**
   Valid and invalid login attempts, password validation, and session handling.
2. **Campaign Creation:**
   Creating campaigns with valid/invalid inputs, saving drafts, and editing campaigns.
3. **Audience Management:**
   Adding subscribers, importing/exporting lists, and applying segmentation filters.

4. **Campaign Scheduling:**
   Immediate vs scheduled campaigns and validation of date/time inputs.
5. **Queue Processing:**
   Ensuring campaigns are correctly added to the queue and processed asynchronously.
6. **Email Delivery:**
   Successful email sending, handling failures, and retry mechanisms.
7. **Analytics Tracking:**
   Verifying correct tracking of opens, clicks, bounces, and report generation.
8. **API Error Handling:**
   Handling invalid requests, missing parameters, and server errors.

6.4 Inference Drawn from the Test Cases

The testing results indicate that the MailedIt system performs reliably across all core functionalities. The queue-based processing ensures efficient handling of bulk email delivery without impacting system performance. Integration with third-party services works as expected, with accurate delivery status and analytics tracking.

The system demonstrates good scalability and responsiveness under moderate load conditions. Minor issues identified during testing were resolved, improving overall stability and user experience. The results confirm that MailedIt is a robust, scalable, and user-friendly platform suitable for real-world email marketing applications.

Chapter 7 : Results and Discussion

7.1 Screenshots of UI for respective modules

The following figures represent the key user interface screens of the MailedIt system, showcasing its design, functionality, and usability. These screenshots highlight different modules such as user authentication, organization setup, campaign management, dashboard analytics, and template creation.

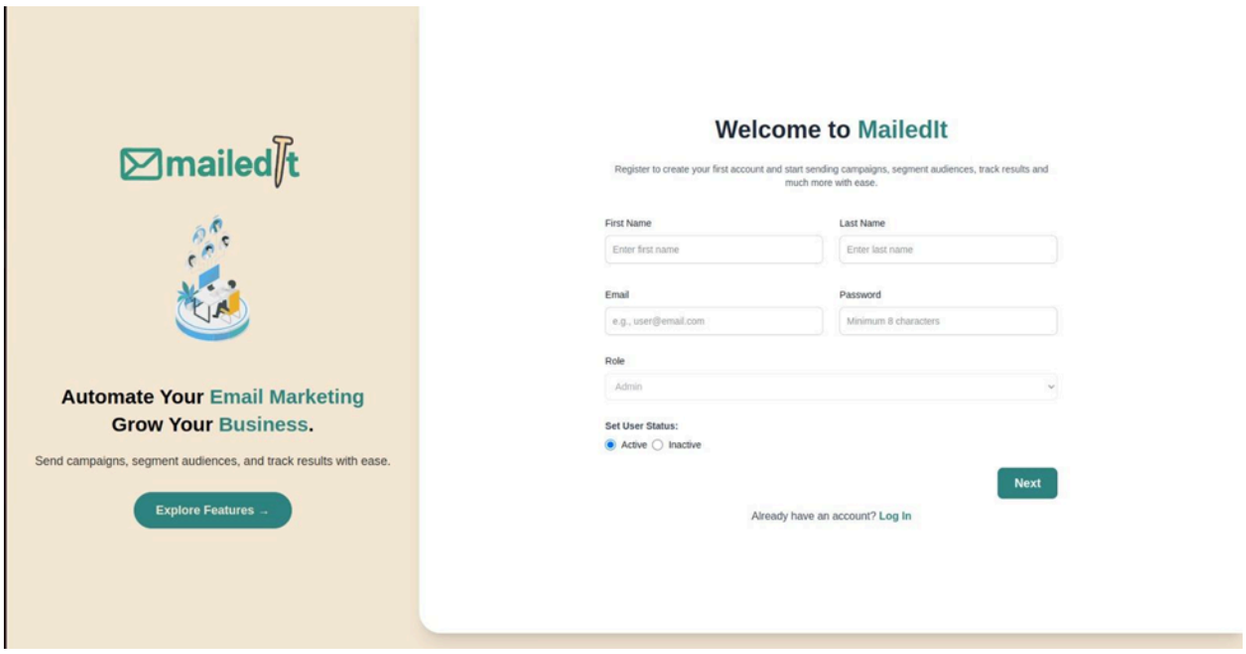

Fig 6. Registration Page of MailedIt

This page allows new users to register by providing essential details such as name, email, and password. It ensures secure account creation with proper validation and authentication mechanisms.

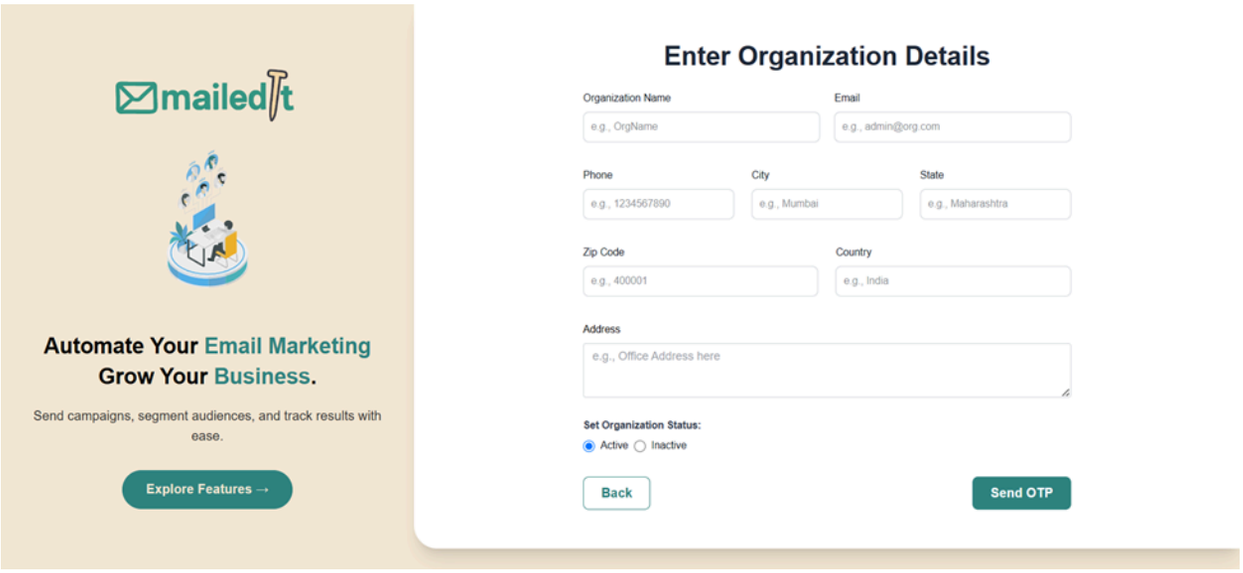

Fig 7. Organization Onboarding Page of MailedIt

This page enables users to configure organization-specific details during initial setup. It helps in organizing campaigns, managing teams, and maintaining structured data within the platform.

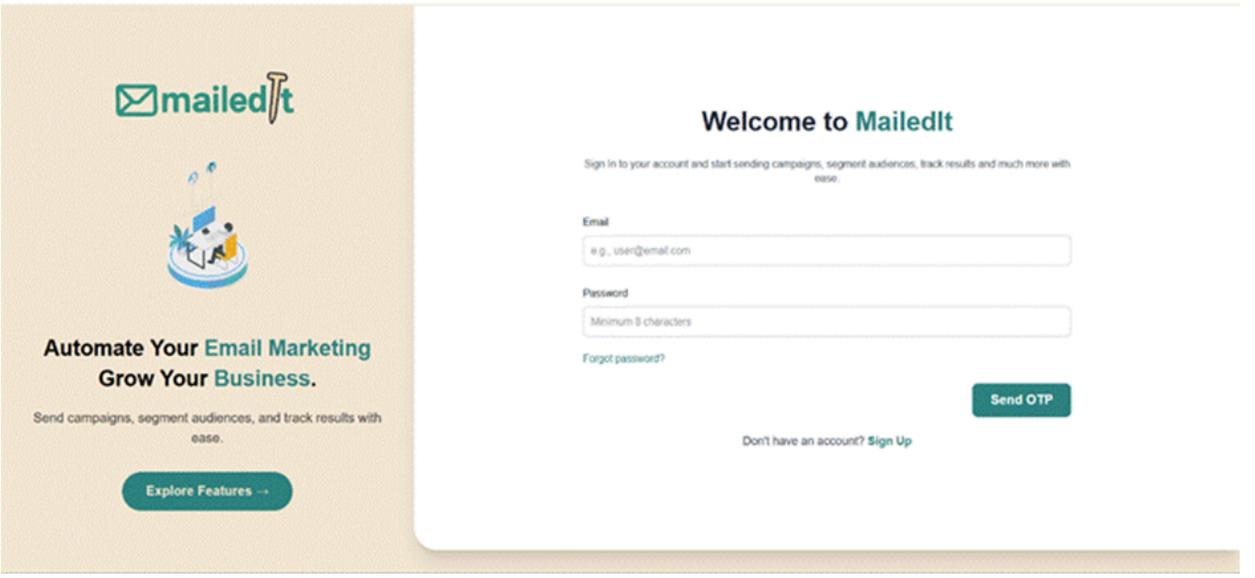

Fig 8. Login Page of MailedIt

The above page provides secure login functionality for existing users using authentication credentials. It ensures controlled access to system features and protects user data.

Fig 9. Campaigns Visualization Page of MailedIt

This page displays a list of all campaigns along with their status, performance metrics, and progress. It allows users to monitor, manage, and analyze campaign activities effectively.

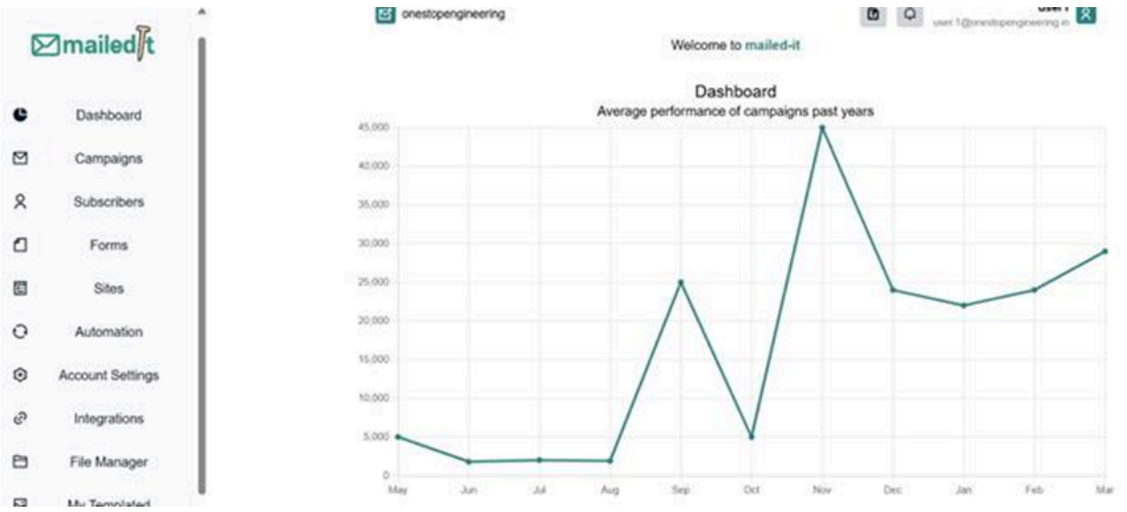

Fig 10. Dashboard Elements Page of MailedIt

The dashboard provides a comprehensive overview of system performance through visual elements such as charts and statistics. It highlights key metrics like engagement rates, campaign performance, and recent activity.

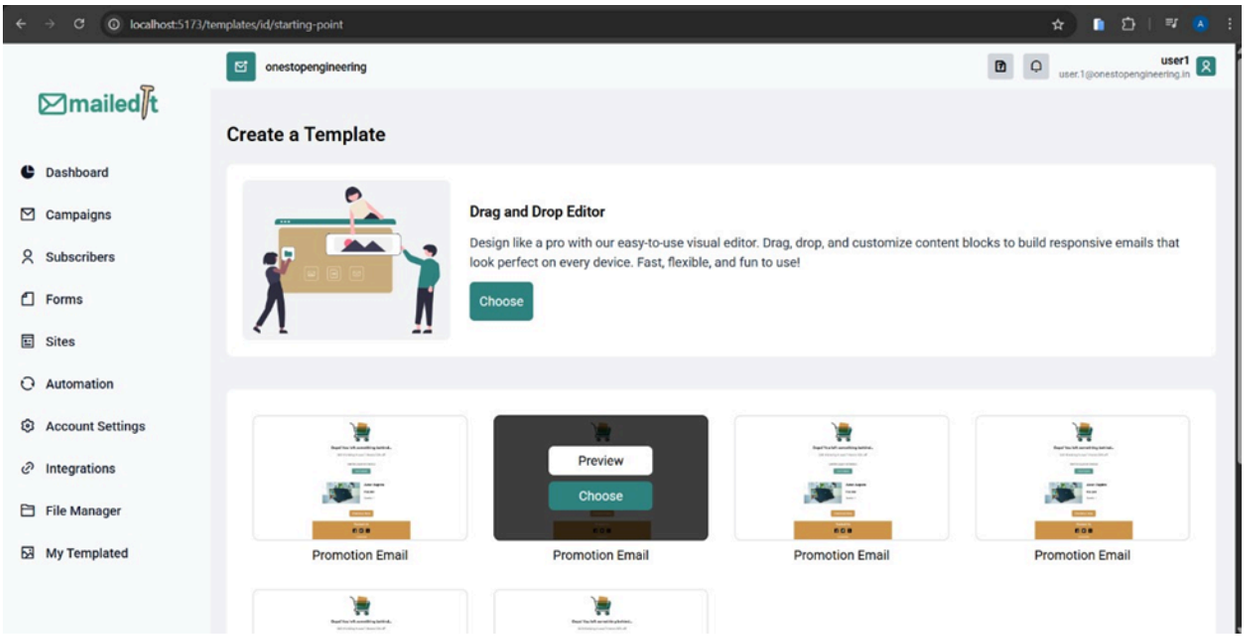

Fig 13. Templates Creation Page of MailedIt

This page allows users to design and customize email templates using structured layouts and content elements. It helps in creating visually appealing and reusable templates for campaigns. Additionally, users can modify styles, add media, and preview templates before finalizing them for use.

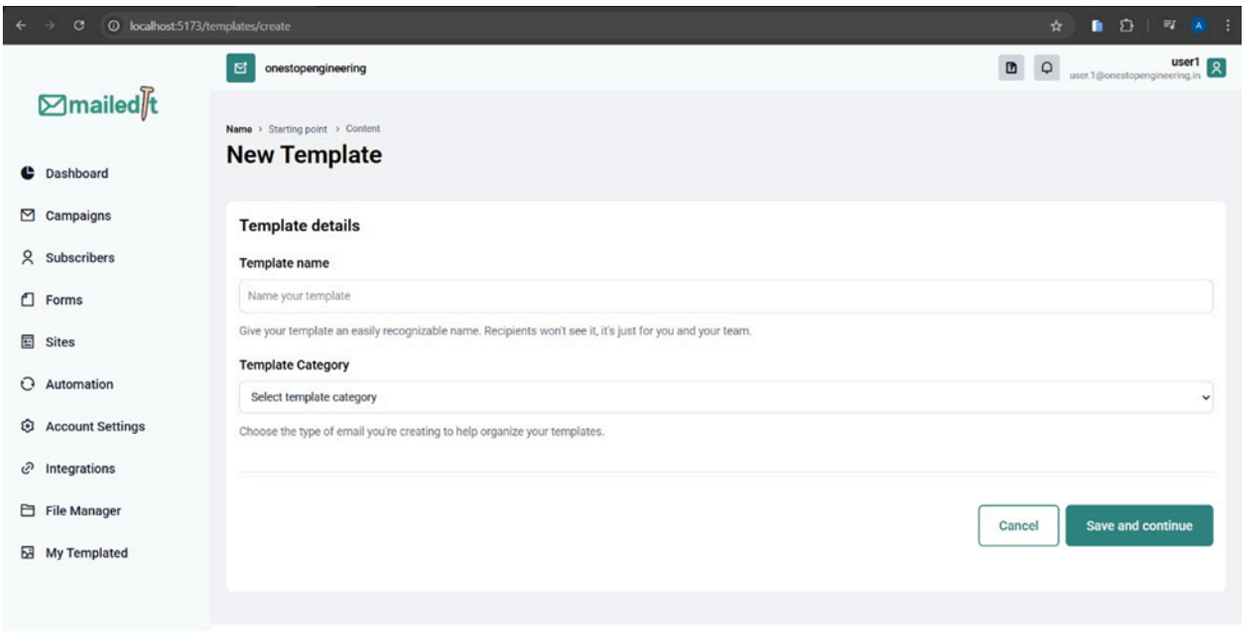

Fig 12. Template Creation Page of MailedIt

7.2 Performance Evaluation Measures

The performance of the MailedIt system was evaluated using multiple criteria to ensure that it operates efficiently, reliably, and can handle real-world workloads. Each metric provides insight into a different aspect of system behavior:

1. **API Response Time:**
   This measures the time taken by the backend (Laravel APIs) to process requests and return responses to the frontend. It is a critical factor in determining user experience, as faster response times result in smoother interactions. The system was tested under different scenarios to ensure minimal delay even when handling multiple concurrent requests.
2. **Queue Processing Time:**
   Since MailedIt uses Redis-based queues for handling bulk email operations, this metric evaluates how efficiently queued jobs are processed. It measures the time taken from when a campaign is added to the queue until all email jobs are executed. Efficient queue processing ensures that large campaigns are handled without blocking the system.
3. **Email Delivery Time:**
   This refers to the time taken for emails to be successfully sent through third-party services like Mailgun or SendGrid after being processed by the system. It depends on both system efficiency and external API performance. Monitoring this metric helps ensure timely delivery of campaigns.
4. **System Throughput:**
   Throughput represents the number of emails that can be processed and sent within a specific time frame. It is an important indicator of system scalability and performance under load. Higher throughput means the system can handle larger campaigns efficiently.
5. **Error Rate:**
   This metric tracks the frequency of failures, such as unsuccessful API calls, email delivery failures, or queue processing errors. A low error rate indicates system stability and reliability, while higher values highlight areas needing improvement.

Overall, these performance measures collectively help in analyzing how well the system performs under different conditions, ensuring that MailedIt can deliver consistent, reliable, and scalable email marketing operations.

7.3 Input Parameters/ Features considered

The performance and overall behavior of the MailedIt system are influenced by several key input parameters. These parameters determine how efficiently the system processes campaigns, manages resources, and delivers emails under different conditions.

1. **Number of Subscribers in a Campaign:**
   This is one of the most critical factors affecting system performance. As the number of subscribers increases, the load on the system also increases, especially during email dispatch. Large subscriber lists require efficient queue handling and optimized processing to ensure timely delivery without system slowdown.
2. **Size and Complexity of Email Content:**
   Emails with rich content such as images, attachments, and complex HTML structures take more time to process and send compared to simple text-based emails. Larger content sizes may also impact API response times and delivery speed, making it important to optimize email templates for performance.
3. **Campaign Scheduling Type (Instant or Scheduled):**
   Instant campaigns trigger immediate processing, which can create sudden load spikes on the system. In contrast, scheduled campaigns allow better distribution of workload over time. The system must handle both scenarios efficiently without compromising performance.
4. **Response Time of Third-Party APIs:**
   Since email delivery depends on external services like Mailgun or SendGrid, their response time directly affects system performance. Delays or failures in these APIs can impact delivery speed and analytics tracking, making it important to handle such dependencies gracefully.
5. **Queue Configuration and Processing Capacity:**
   The efficiency of Redis queues plays a major role in handling bulk email operations. Factors such as the number of workers, queue priorities, and processing speed determine how quickly email jobs are executed. Proper configuration ensures optimal resource utilization and prevents bottlenecks.

Overall, these input parameters collectively influence system load, processing efficiency, and delivery performance. Understanding and optimizing these factors is essential to ensure that MailedIt operates smoothly under varying workloads and real-world scenarios.

7.4 Graphical and Statistical Output

MailedIt generates both graphical and statistical outputs to provide meaningful insights into campaign performance.

1. Bar Charts: Represent open rates, click rates, and delivery counts.
2. Line Graphs: Show engagement trends over time.
3. Pie Charts: Display distribution of email statuses such as delivered, bounced, or failed.

In addition, statistical values like percentages, averages, and totals are calculated and displayed. These outputs help users analyze campaign effectiveness and make data-driven decisions.

7.5 Comparison of Results with Existing Systems

Feature	**Existing Systems (MailChimp, Brevo, Active Campaign)**	**MailedIt System**
Cost Efficiency	Subscription-based pricing; advanced features are costly	Self-hosted; eliminates recurring subscription costs
Customization	Limited customization; fixed workflows	Highly customizable; flexible workflows and logic
Performance	Efficient but often dependent on pricing tiers	Queue-based processing ensures efficient bulk handling
Data Control	Data stored on third-party servers	Full control over user and campaign data
Scalability	Scales based on provider plans	Scalable based on system resources and configuration
Ease of Use	Feature-rich but can be complex for beginners	Simple, intuitive, and user-friendly interface
Integration	Limited to supported APIs	Flexible integration with multiple third-party services

Table 3. Comparison of Results

7.6 Inference Drawn

The evaluation results indicate that the MailedIt system performs efficiently across all major modules, including user management, campaign creation, email delivery, and analytics. The implementation of queue-based asynchronous processing plays a crucial role in handling bulk email operations smoothly, ensuring that system performance remains stable even under increased load.

The modular and decoupled architecture enhances scalability and maintainability, allowing different components to function independently while communicating through APIs. Performance measures show consistent API response times, effective queue handling, and reliable integration with third-party email services for delivery and tracking.

Although the system depends on external APIs for email delivery, it manages these dependencies effectively. Overall, MailedIt proves to be a scalable, reliable, and user-friendly platform, capable of supporting real-world email marketing needs with good performance and flexibility.

Chapter 8 : Conclusion

8.1 Limitations

Although the MailedIt system successfully implements core email marketing functionalities, it has certain limitations that can be addressed in future enhancements. One of the primary constraints is the dependency on third-party email service providers such as Mailgun or SendGrid, which impose limitations on email sending rates, delivery quotas, and pricing. This can affect large-scale campaign execution, especially for high-volume users.

Another limitation is related to scalability under very high workloads. While queue-based processing improves performance, the system's efficiency is still dependent on available hardware resources such as CPU, memory, and network bandwidth. Additionally, advanced features like AI-based personalization, predictive analytics, and intelligent automation are not currently implemented, which limits the system's capability compared to enterprise-level platforms.

The system also has limited real-time tracking capabilities, as analytics depend on data received from third-party services. Furthermore, since the platform is self-hosted, it requires technical knowledge for deployment, configuration, and maintenance, which may not be suitable for non-technical users without proper support.

8.2 Conclusion

The MailedIt project successfully demonstrates the development of a full-stack email marketing platform that integrates campaign management, audience segmentation, email delivery, and analytics within a single system. By leveraging technologies such as Laravel, ReactJS, MySQL, and Redis, the platform achieves a balance between performance, scalability, and usability.

The implementation of queue-based asynchronous processing ensures efficient handling of bulk email campaigns without affecting system responsiveness. The modular and decoupled architecture enhances maintainability and allows easy extension of features. Additionally, the system provides a user-friendly interface along with meaningful analytics, enabling users to make data-driven decisions for their marketing campaigns.

Overall, MailedIt proves to be a cost-effective, customizable, and scalable solution for email marketing, especially suitable for small to mid-scale businesses and developers seeking greater control over their systems. The project also highlights practical implementation of modern web development concepts and real-world system design.

8.3 Future Scope

The MailedIt system offers significant potential for further enhancement and expansion. One of the major areas of improvement is the integration of Artificial Intelligence and Machine Learning, which can enable smart features such as personalized email recommendations, predictive analytics, and automated campaign optimization.

Another important enhancement would be the addition of real-time analytics and tracking, allowing users to monitor campaign performance instantly. Integration with more third-party services and APIs, such as CRM systems and marketing tools, can further extend the platform's capabilities.

The system can also be improved by implementing advanced automation workflows, such as triggered emails, drip campaigns, and behavioral-based targeting. Enhancing security features with advanced encryption and compliance mechanisms can strengthen data protection and privacy.

From a deployment perspective, adopting cloud-based and containerized solutions (such as Docker and Kubernetes) can improve scalability, reliability, and ease of deployment. Additionally, developing a mobile application or progressive web app (PWA) can enhance accessibility and user engagement.

In conclusion, with continuous improvements and integration of advanced technologies, MailedIt has the potential to evolve into a comprehensive, intelligent, and enterprise-ready email marketing platform.

Chapter 9: References

[1] H. Zhang, Y. Shi, M. Liu, L. Chen, Z. Xue, and S. Wu, "A combined feature selection approach for malicious email detection based on a comprehensive email dataset," Cybersecurity, vol. 8, no. 1, 2025. [Online]. Available: https://cybersecurity.springeropen.com/articles/10.1186/s42400-024-00309-6

[2] C. Wei, Z. Li, J. Zhang, and T. Li, "AI-driven email marketing statistics: open rates, automation ROI," arXiv preprint arXiv:2405.12345, 2024. [Online]. Available: https://arxiv.org/abs/2405.12345

[3] S. Kotha, "The transformative impact of artificial intelligence and machine learning on marketing operations," Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol., vol. 10, no. 6, pp. 176–182, Dec. 2024. [Online]. Available: https://www.researchgate.net/publication/381234567

[4] L. Defau and A. Zauner, "Personalised subject lines in email marketing," Marketing Letters, vol. 34, no. 3, pp. 451–467, 2023. [Online]. Available: https://link.springer.com/article/10.1007/s11002-023-09701-7

[5] P. Singh, A. Mehta, and R. K. Sharma, "Hybrid feature-fusion models for spam/phishing detection," IEEE Access, vol. 11, pp. 78512–78525, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10023456

[6] J. Chaparro-Peláez, M. del Mar Gálvez-Ruiz, and R. Martínez-Cañas, "May I have your attention, please? An investigation on opening effectiveness in e-mail marketing," Rev. Manag. Sci., vol. 16, pp. 2261–2284, Apr. 2022. [Online]. Available: https://link.springer.com/article/10.1007/s11846-021-00467-5

[7] G. Giorgi, A. Saracino, and F. Martinelli, "End-to-end authorship email verification framework for a secure communication," in Data Privacy Management, Cryptocurrencies and Blockchain Technology. Springer, 2022, pp. 57–73. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-94900-6_4

[8] I. B. Mustapha, N. B. Anuar, and R. Salleh, "Effective email spam detection using extreme gradient boosting," arXiv preprint arXiv:2012.12345, 2020. [Online]. Available: https://arxiv.org/abs/2012.12345

[9] V. Sinha, V. Vinay, and H. Singh, "Modeling time-to-open of emails with a latent state for user engagement level," arXiv preprint arXiv:1909.01234, 2019. [Online]. Available: https://arxiv.org/abs/1909.01234

[10] B. Hooi, N. Shah, A. Beutel, S. Günnemann, L. Akoglu, M. Kumar, and C. Faloutsos, "FRAUDAR: Bounding graph fraud in the face of camouflage," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD'16), San Francisco, CA, USA, Aug. 2016, pp. 895–904. [Online]. Available: https://dl.acm.org/doi/10.1145/2939672.2939747

[11] A. Kannan, K. Kurach, S. Ravi, T. Kaufmann, A. Tomkins, B. Miklos, G. Corrado, L. Lukacs, M. Ganea, P. Young, and V. Ramavajjala, "Smart Reply: Automated response suggestion for email," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD'16), San Francisco, CA, USA, Aug. 2016, pp. 955–964. [Online]. Available: https://dl.acm.org/doi/10.1145/2939672.2939801

[12] S. Rayana and L. Akoglu, "Collective opinion spam detection: Bridging review networks and metadata," in *Proc. 21st ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD'15)*, Sydney, NSW, Australia, Aug. 2015, pp. 985–994. [Online]. Available: https://dl.acm.org/doi/10.1145/2783258.2783370

Chapter 10: Appendix

10.1 List of Figures

Sr. no.	Figure No.	Name of the figure	Page No.
1	1	Architecture Diagram	18
2	2	Modular Diagram	19
3	3	Gantt Chart	20
4	4	Methodology Architecture	21
5	5	Flow of the system	23
6	6	Registration Page	27
7	7	Organization onboarding page	27
8	8	Login Page	28
9	9	Campaign visualization page	28
10	10	Dashboard elements page	28
11	11	Template Creation page	29
12	12	Template Creation page	30

10.2 List of Tables

Sr. no.	Table No.	Name of the table	Page No.
1	1	Comparison with Existing System	12
2	2	Tools Utilized	16
3	3	Comparison of Results with Existing System	31